Solution: Kth Smallest Number in a Sorted Matrix

Problem Statement

Given an N * N matrix where each row and column is sorted in ascending order, find the Kth smallest element in the matrix.

Example 1:

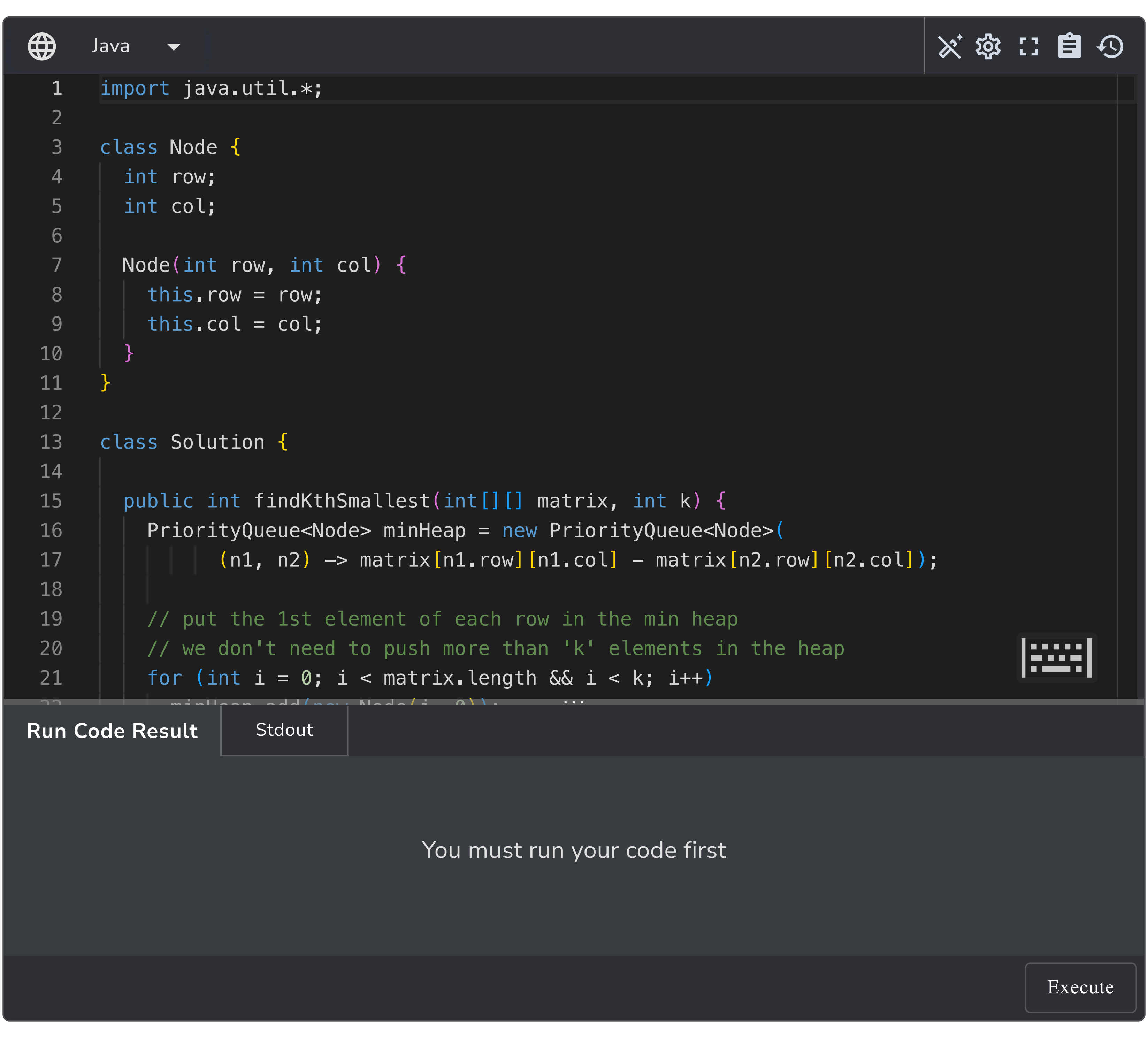
```
Input: Matrix=[
    [2, 6, 8], 
    [3, 7, 10],
    [5, 8, 11]
  ], 
  K=5
Output: 7
Explanation: The 5th smallest number in the matrix is 7.
```

Solution

This problem follows the **K-way merge** pattern and can be easily converted to `Kth Smallest Number in M Sorted Lists`. As each row (or column) of the given matrix can be seen as a sorted list, we essentially need to find the Kth smallest number in 'N' sorted lists.

Code

Here is what our algorithm will look like:



```java
import java.util.*;

class Node {
  int row;
  int col;

  Node(int row, int col) {
    this.row = row;
    this.col = col;
  }
}

class Solution {

  public int findKthSmallest(int[][] matrix, int k) {
    PriorityQueue<Node> minHeap = new PriorityQueue<Node>(
        (n1, n2) -> matrix[n1.row][n1.col] - matrix[n2.row][n2.col]);

    // put the 1st element of each row in the min heap
    // we don't need to push more than 'k' elements in the heap
    for (int i = 0; i < matrix.length && i < k; i++)
```

Time Complexity

First, we inserted at most 'K' or one element from each of the 'N' rows, which will take O(min(K,N)). Then we went through at most 'K' elements in the matrix and remove/add one element in the heap in each step. As we can't have more than 'N' elements in the heap in any condition, therefore, the overall time complexity of the above algorithm will be O(min(K, N) + K*logN).

Space Complexity

The space complexity will be O(N) because, in the worst case, our min-heap will be storing one number from each of the 'N' rows.

An Alternate Approach

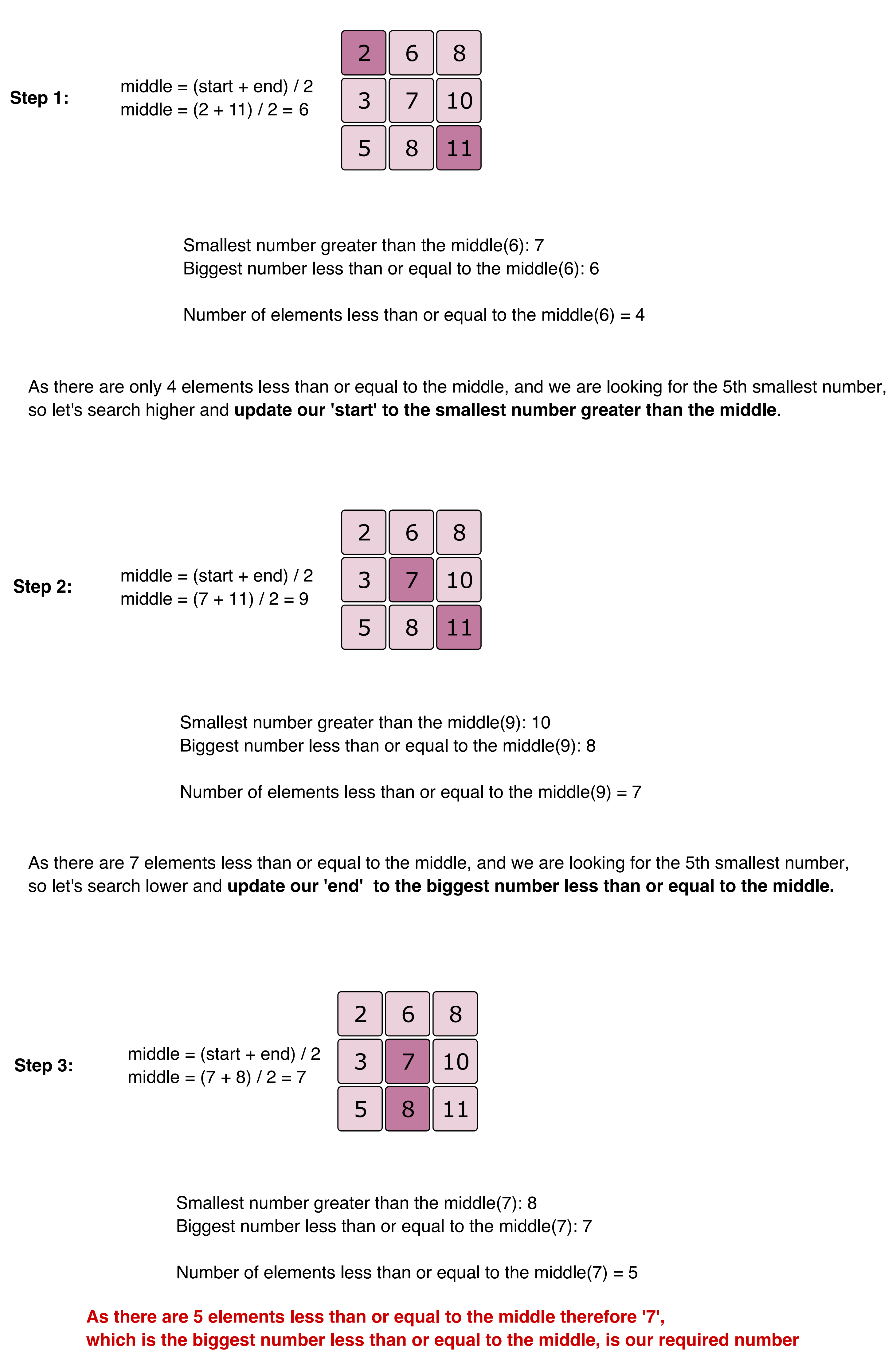
Since each row and column of the matrix is sorted, is it possible to use **Binary Search** to find the Kth smallest number?

The biggest problem to use **Binary Search**, in this case, is that we don't have a straightforward sorted array, instead, instead, we have a matrix! As we remember, in **Binary Search**, we calculate the middle index of the search space ('1' to 'N') and see if our required number is pointed out by the middle index; if not we either search in the lower half or the upper half. In a sorted matrix, we can't really find a middle. Even if we do consider some index as middle, it is not straightforward to find the search space containing numbers bigger or smaller than the number pointed out by the middle index.

An alternative could be to apply the **Binary Search** on the "number range" instead of the "index range". As we know that the smallest number of our matrix is at the top left corner and the biggest number is at the bottom right corner. These two numbers can represent the "range" i.e., the start and the end for the Binary Search. Here is how our algorithm will work:

1. Start the **Binary Search** with `start = matrix[0][0]` and end = `matrix[n−1][n−1]`.
2. Find middle of the `start` and the `end`. This `middle` number is NOT necessarily an element in the matrix.
3. Count all the numbers smaller than or equal to `middle` in the matrix. As the matrix is sorted, we can do this in O(N).
4. While counting, we can keep track of the "smallest number greater than the `middle`" (let's call it `n1`) and at the same time the "biggest number less than or equal to the middle" (let's call it n2). These two numbers will be used to adjust the "number range" for the Binary Search in the next iteration.
5. If the count is equal to 'K', `n2` will be our required number as it is the "biggest number less than or equal to the `middle`", and is definitely present in the matrix.
6. If the count is less than 'K', we can update `start = n2` to search in the higher part of the matrix and if the count is greater than 'K', we can update `end = n1` to search in the lower part of the matrix in the next iteration.

Here is the visual representation of our algorithm:


Step 1:
middle = (start + end) / 2
middle = (2 + 11) / 2 = 6
2 6 8
3 7 10
5 8 11
Step 2:
middle = (start + end) / 2
middle = (7 + 11) / 2 = 9
2 6 8
3 7 10
5 8 11
Step 3:
middle = (start + end) / 2
middle = (7 + 8) / 2 = 7
2 6 8
3 7 10
5 8 11

Smallest number greater than the middle(6): 7
Biggest number less than or equal to the middle(6): 6

Number of elements less than or equal to the middle(6) = 4

As there are only 4 elements less than or equal to the middle, and we are looking for the 5th smallest number, so let's search higher and **update our 'start' to the smallest number greater than the middle**.

Smallest number greater than the middle(9): 10
Biggest number less than or equal to the middle(9): 8

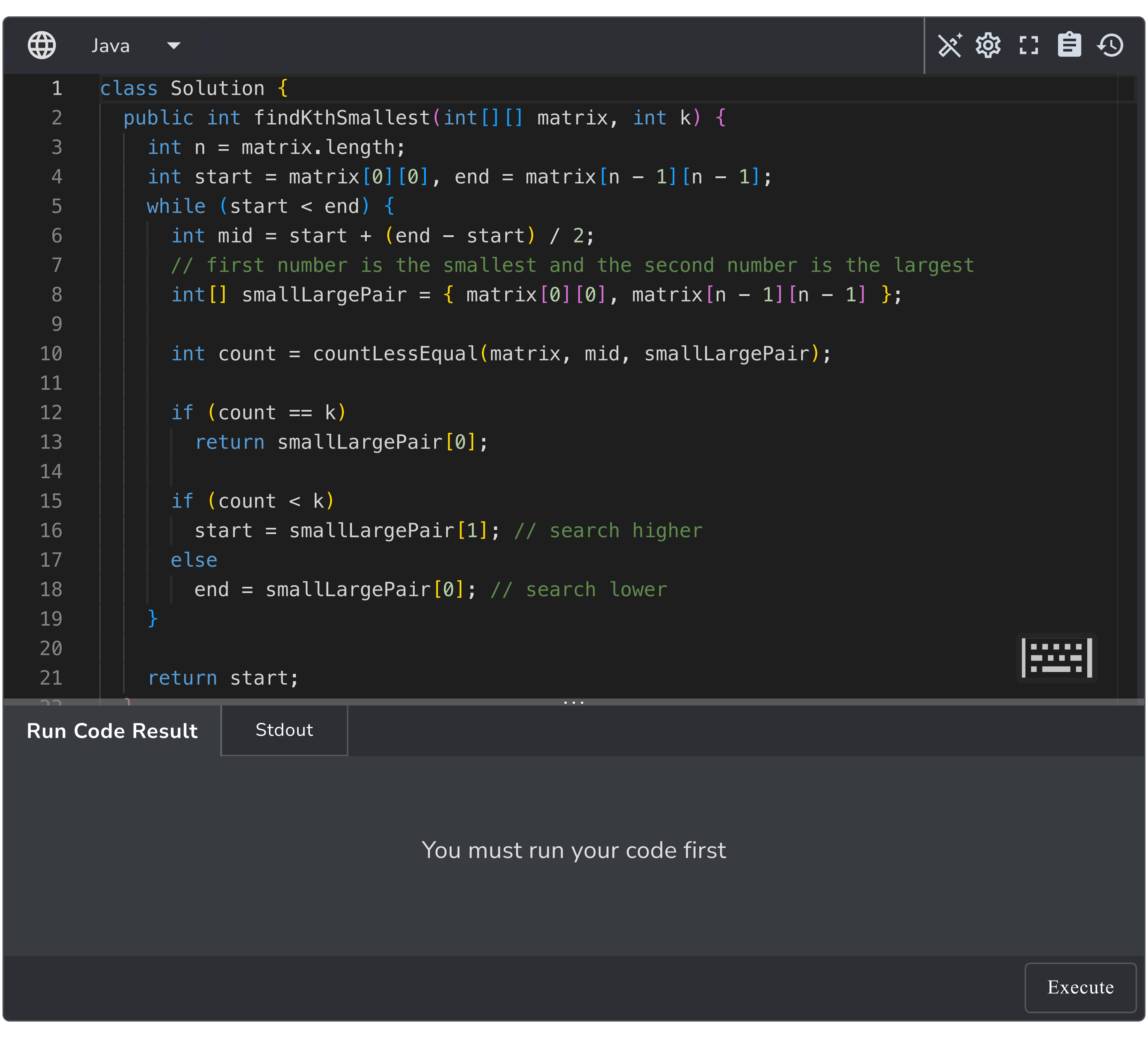
Number of elements less than or equal to the middle(9) = 7

As there are 7 elements less than or equal to the middle, and we are looking for the 5th smallest number, so let's search lower and **update our 'end' to the biggest number less than or equal to the middle.**

Smallest number greater than the middle(7): 8
Biggest number less than or equal to the middle(7): 7

Number of elements less than or equal to the middle(7) = 5

As there are 5 elements less than or equal to the middle therefore '7', which is the biggest number less than or equal to the middle, is our required number



```java
class Solution {
  public int findKthSmallest(int[][] matrix, int k) {
    int n = matrix.length;
    int start = matrix[0][0], end = matrix[n - 1][n - 1];
    while (start < end) {
      int mid = start + (end - start) / 2;
      // first number is the smallest and the second number is the largest
      int[] smallLargePair = { matrix[0][0], matrix[n - 1][n - 1] };

      int count = countLessEqual(matrix, mid, smallLargePair);

      if (count == k)
        return smallLargePair[0];

      if (count < k)
        start = smallLargePair[1]; // search higher
      else
        end = smallLargePair[0]; // search lower
    }

    return start;
```

Time Complexity

The Binary Search could take (log(max−min)) iterations where 'max' is the largest and 'min' is the smallest element in the matrix and in each iteration we take O(N) for counting, therefore, the overall time complexity of the algorithm will be O(N*log(max-min)).

Space Complexity

The algorithm runs in constant space O(1).

← Previous
Kth Smallest Number In A Sorted Matrix (Hard)
Next →
Smallest Number Range (Hard)
☐ Mark as Completed